

1. (8 points) The 32-bit instruction below is encoded in hex as 0x007302B3 (binary: 0000 0000 0111 0011 0000 0010 1011 0011). Identify the instruction format, break it into its fields, and write the equivalent RISC-V assembly instruction.

opcode [6:0] = 0110011 = 0x33 → R-type

funct3 [14:12] = 000 → arithmetic family

funct7 [31:25] = 0000000 → ADD

rs1 [19:15] = 00000 = x0

rs2 [24:20] = 00011 = x3

rd [11:7] = 01010 = x10

Instruction: add x10, x0, x3

2. (8 points) Encode the instruction `addi x10, x11, -5` into its 32-bit binary representation and give the result in hex. Show the value of each field.

`addi x10, x11, -5`

0xFFB50513: binary 11111111 1011 0101 1000 0101 0001 0011

opcode [6:0] = 0010011 = 0x13 I-type

funct3 [14:12] = 000 ADDI

imm[11:0] [31:20] = 111111111011 = -5

rs1 [19:15] = 01011 = x11

rd [11:7] = 01010 = x10

Instruction: `addi x10, x11, -5`

3. (8 points) The instruction 0x0084A623 is given in hex. Identify the instruction format, decode its fields, and write the equivalent assembly instruction.

0x0084A623: binary 00000000 1000 0100 1010 0110 0010 0011

opcode [6:0] = 0100011 = 0x23 S-type

funct3 [14:12] = 010 Store word (sw)

rs1 [19:15] = 10100 = x20 Base address register

rs2 [24:20] = 10000 = x16 Source register (value to store)

imm[11:5] [31:25] = 00000000

imm[4:0] [11:7] = 00110 = 6

Combined immediate: 00000000 00110 = 6

Instruction: `sw x16, 6(x20)`

4. (8 points) The instruction 0xFEC12703 is given in hex. Identify the instruction format, decode its fields (note the immediate is negative), and write the equivalent assembly instruction.

0xFEC12703: binary 11111110 1100 0001 0010 0111 0000 0011

opcode [6:0] = 0000011 = 0x03 I-type (Load)
funct3 [14:12] = 010 Load word (lw)
rs1 [19:15] = 00001 = x1 Base address register
rd [11:7] = 01110 = x14 Destination register
imm[11:0] [31:20] = 11111110110 = -10

Instruction: lw x14, -10(x1)

5. (8 points) (a) (4 points) Decode the instruction 0xFE321C23 into assembly.
(b) (4 points) Encode `sub x18, x19, x20` into 32-bit binary and hex.

0xFE321C23: binary 11111110 0011 0010 0001 1100 0010 0011

opcode [6:0] = 0100011 = 0x23 S-type (Store)
funct3 [14:12] = 000 Store byte (sb)
rs1 [19:15] = 00011 = x3 Base address register
rs2 [24:20] = 10011 = x19 Source register (value to store)
imm[11:5] [31:25] = 1111111
imm[4:0] [11:7] = 11000 = 24

Combined immediate: 1111111 11000 = -32 (in two's complement)

Instruction: sb x19, -32(x3)

(b):sub x18, x19, x20

Format: R-type (Register)
Opcode: 0x33 (binary: 0110011)
funct3: 000 (Arithmetic)
funct7: 0100000 (SUB operation)

Operands:

rd (destination): x18 = register 18
rs1 (source 1): x19 = register 19
rs2 (source 2): x20 = register 20

Binary fields:

funct7 [31:25] = 0100000
rs2 [24:20] = 10100 = x20
rs1 [19:15] = 10011 = x19
funct3 [14:12] = 000
rd [11:7] = 10010 = x18
opcode [6:0] = 0110011

Full 32-bit binary:

01000001010010011000100100110011

Group into 4-bit nibbles:

0100 0010 1001 0011 1000 1001 0011 0011

Convert to hex:

4 2 9 3 8 9 3 3

Hex: 0x42938933

Binary: 010000010100100111000100100110011

Instruction: sub x18, x19, x20

6. (8 points) Convert the decimal value **-22.5** to IEEE-754 single precision. Show the sign, exponent, and mantissa bits, then give the final answer in hex.

Sign bit = 1 (-)

Convert 22.5 to binary

22 = 10110

0.5 = .1

22.5 = 10110.1

Normalize to scientific notation

$10110.1 = 1.01101 \times 2^4$

Exponent = 4

Calculate biased exponent

Bias for single precision = 127

Biased exponent = $4 + 127 = 131 = 10000011$

Extract mantissa (fractional part after the 1)

Mantissa = 01101 (from 1.01101)

Padded to 23 bits = 01101000000000000000000

Assemble IEEE-754 format

Sign bit [31] = 1

Exponent [30:23] = 10000011

Mantissa [22:0] = 01101000000000000000000

Full 32-bit binary:

1 10000011 01101000000000000000000

Group into nibbles:

1100 0011 0110 1000 0000 0000 0000 0000

Convert to hex:

C 3 6 8 0 0 0 0

Final answer: 0xC3680000

7. (8 points) Convert the IEEE-754 single precision hex value 0xC1500000 back to decimal. Show your work (sign, exponent, mantissa, and the reconstructed value).

Convert hex to binary

0xC1500000 = 1100 0001 0101 0000 0000 0000 0000 0000

Extract fields

Sign bit [31] = 1

Exponent [30:23] = 10000010

Mantissa [22:0] = 101000000000000000000000

Interpret sign bit

Sign = 1, so number is negative

exponent value

Biased exponent = 10000010 = 130

Bias for single precision = 127

Actual exponent = 130 - 127 = 3

mantissa

Mantissa bits = 101000000000000000000000

Insert implicit leading 1: 1.101 (from mantissa 101)

Calculate decimal value

$1.101 \times 2^3 = 1101$ (binary) = 13 (decimal)

Apply sign

Result = -13

Final answer: -13.0

8. (8 points) Convert the decimal value 0.1 to IEEE-754 single precision. (Note: 0.1 has no exact binary representation — round to 23 mantissa bits and note that the result is an approximation.)

Determine sign bit

Positive sign bit = 0

Convert 0.1 to binary (fractional)

0.1 (decimal) = 0.00011001100110011

scientific notation

$0.00011001100110011... = 1.1001100110011... \times 2^{-4}$

Exponent = -4

Calculate biased exponent

Bias for single precision = 127

Biased exponent = -4 + 127 = 123 = 01111011

Extract mantissa (23 bits, rounded)

Mantissa from 1.1001100110011... = 10011001100110011001100 (23 bits)

Round to nearest: 1001100110011001100110

(7 points) What is the purpose of the Program Counter (PC) in a RISC-V processor? How does its value change during normal sequential instruction execution versus during a branch or jump instruction?

- The program counter is a register that holds the memory address of the instruction that is being used currently (fetched or executed)
- During Normal sequential instruction execution, the PC increments by 4 ($PC = PC + 4$) as one RISC-V instruction is 4 bytes. Instructions execute in the order they sit in memory.
- During Branch
 - o if the condition is false, branch is not taken ($PC = PC + 4$)
 - o If true: $PC = PC + \text{offset}$, and the offset is the sign extended immediate from the instruction
 - o This makes it so branches can jump forward or backwards from current instruction
- Jump Instructions
 - o `jal rd, label`: $PC = PC + \text{offset}$
 - o `jalr rd, offset(rs1)`: $PC = rs1 + \text{offset}$ and saves $PC + 4$ in `rd`

(c) (6 points) What is the purpose of the stack pointer (register `x2 / sp` in RISC-V)? How does its value change when data is pushed onto or popped off the stack?

- Stack pointer holds the memory address of the top of the stack which last pushed an item, used for saved registers, return address, and local variables
- Pushing Data
 - o The stack grows downwards toward lower memory addresses
 - o `SP` is decremented to make room, and stored from offsets
- Popping data:
 - o The values are loaded back from their offsets
 - o `SP` is incremented by the same amount of the free space
 - o the data is not erased but stays in memory. It is treated as free space and is overwritten by later pushes